

Кэширование методов бизнес-логики

Участок: Кэширование

Кэширование — это процесс сохранения данных в специально отведенном месте (промежуточном хранилище) для ускоренного доступа к ним в будущем.

Кэширование применяется для методов бизнес-логики (**только read-only**) как сохранение результатов вызова метода с заданными значениями аргументов. Это позволяет освободить сервера БД и БЛ от лишней нагрузки, ускорить работу метода и увеличить производительность веб-приложения.

Типовые сценарии использования кэширования:

1. Пользователь часто запрашивает данные, значение которых меняется достаточно редко; **Пример:** персональные данные преподавателей интернет-школы, перечень товаров в прайс-листе интернет-магазина или итоговый отчет деятельности предприятия (например, за месяц);
2. Пользователь совершает ресурсоёмкий запрос, занимающий значительное время; **Пример:** приложение обрабатывает большие объемы данных, используя сложную логику, и затем возвращает данные в качестве отчета, к которому часто обращаются пользователи.

Основной показатель эффективности системы кэширования — коэффициент попаданий запросов в кэш (отношение количества попаданий в кэш к общему количеству запросов). Частые сбросы кэша или кэширование редко запрашиваемых данных не повышают эффективность.

Для хранения кэшированных данных (результатов исполнения методов) используются локальные и облачные хранилища. Выбор используемого хранилища определяется настройками сервиса приложения.

Для использования локального хранилища в настройках необходимо задать:

- 1 **[Ядро.Кэширование методов]**
- 2 `Хранилище=srvlocal`

и подключить к своему сервису модуль BL Cache Local — драйвер локального хранилища для кэширования методов.

Для использования облачного хранилища в настройках необходимо задать:

- 1 **[Ядро.Кэширование методов]**
- 2 `Хранилище=service: cachestorage`

где `cachestorage` — системное имя сервиса кэширования в облаке.

Также к создаваемому сервису требуется подключить сервисный модуль BL Cache Service — драйвер, который обращается к облачному сервису кэширования.



При обновлении сервиса (смене версии) кэш методов сбрасывается.

Сервисные модули BL Cache и BL Cache Service входят в состав [шаблона](#) `basic_platform.s3srv` и доступны при наследовании сервиса от этого шаблона.

Схема процесса кэширования

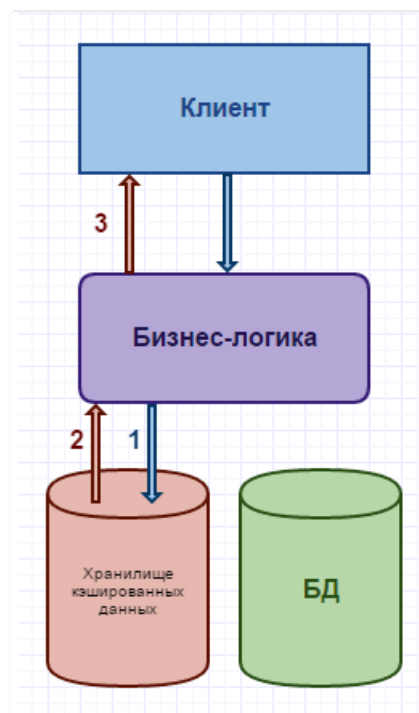
Процесс кэширования включает в себя следующие этапы:

- При первом вызове кэшируемого метода бизнес-логики (кэшируемые данные отсутствуют в хранилище):
 1. Проверка наличия соответствующего поля в хранилище кэшированных данных;
 2. Полноценное выполнение метода с запросом в БД;
 3. Возвращение результата в клиентскую часть веб-приложения и его кэширование в хранилище.



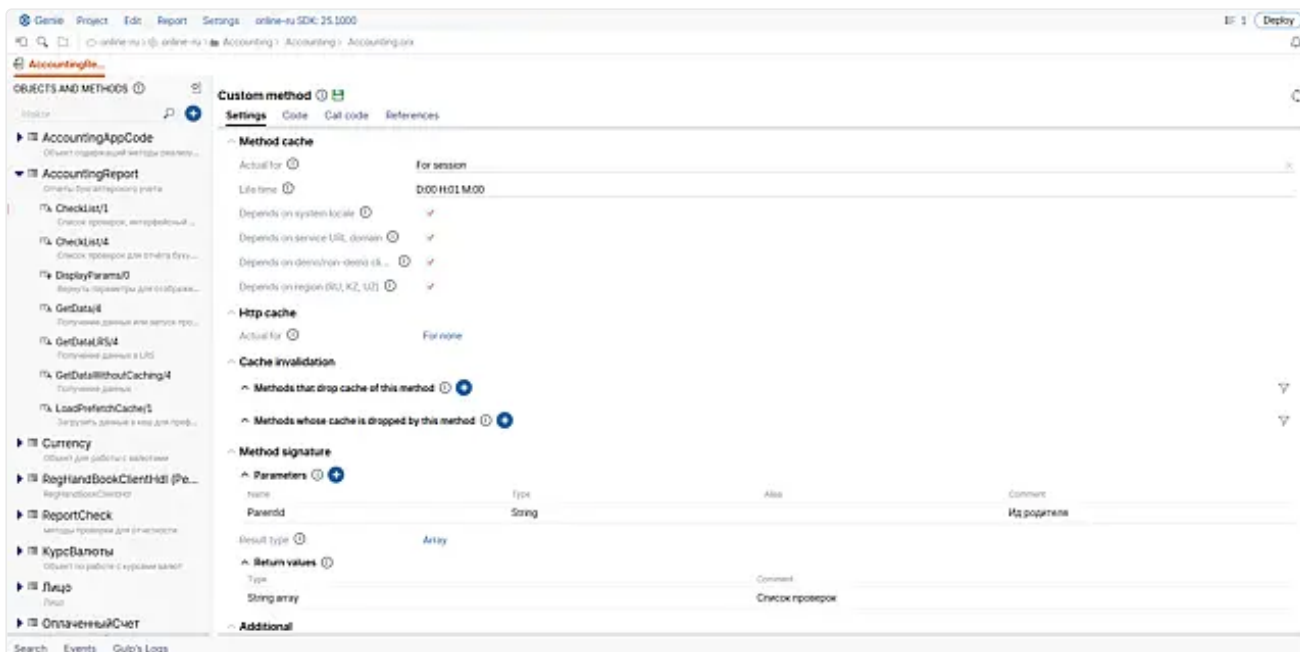
- При последующих вызовах кэшируемого метода бизнес-логики (кэшируемые данные сформированы):

1. Проверка наличия соответствующего поля в хранилище кэшированных данных;
2. Выдача готового ответа, сохраненного в хранилище;
3. Возвращение клиенту результата, сохраненного в хранилище.



Настройка кэширования метода БЛ

Конфигурация выполняется в [Genie](#). В описании метода в блоке **Method cache** укажите настройки кэширования:



Настройки кэширования метода в Genie

Параметры	Описание
Actual for	Область актуальности кэшированных данных, которая определяет для кого они будут доступны. Параметр принимает следующие значения: For none (никому) - кэширования данных не происходит. For session (для сессии). Кэширует данные для пользователя в пределах его сессии. Например, подходит для методов фильтрации по заданным параметрам в рамках одного списка. For user (для пользователя) - кэширует данные для пользователя независимо от его сессии. For person (для персоны) - кэширует данные для персоны независимо от сессии/пользователя/клиента. For client (для клиента) - кэширует данные для всех пользователей клиента. Например, подходит для методов, возвращающих контент веб-приложения, соответствующий определённой группе пользователей. For all (для всех) - кэширует данные для всех клиентов, их пользователей и персон. Например, подходит для методов, отображающих общие интерфейсные элементы навигации. For one RPC (для одного RPC-вызова) - кэширует данные для кода, выполняемого в рамках одного RPC-вызова. Области актуальности For session, For user, For client и For all вложены одна в другую. Сессия - самая узкая область актуальности, соответствует открытой сессии в браузере - соответственно, она завершается при выходе пользователя из системы или может устаревать в случае длительного отсутствия активности. Идентифицируется по ID сессии (icsSESSION_ID). Пользователь - область актуальности, соответствующая пользователю, зарегистрированному в системе. Идентифицируется по icsX_UID.

	• Клиент – область актуальности, соответствующая организации, к которой относится пользователь. Идентифицируется по icsX_CID. • Для всех – глобальная область актуальности, охватывает всё.
Life time	Время жизни кэша – период времени, в течение которого будут храниться кэшированные данные. Параметр может принимать значения от 1 мин до 48 часов и задается в формате DD:HH:MM (D – день. H – час. M – минута). Выбор периода хранения данных должен соответствовать частоте вызова метода с различными значениями аргументов. Если частоту вызова предугадать трудно, установите максимальное время жизни кэша.
Depends on system locale	Если флаг отмечен, то результат вызова метода зависит от системной локали (используемого языка; он может быть переопределен в данных сессии); для различных вариантов выбора языка кэшируются разные данные. При снятии отметки возвращаемый результат от локали не зависит; при изменении языка кэшируемые данные не меняются.
Depends on service URL domain	Если флаг отмечен, то результат вызова метода предполагается зависящим от домена URL сервиса (например, private-cloud.sbis.ru или private.sbis.ru); для различных доменов должны кэшироваться разные данные. При снятии отметки возвращаемый результат от домена не зависит. Зависимость от пути в URL сервиса (например, /srv-name/service/) в любом случае сохраняется.
Depends on demo/non-demo client	Если флаг отмечен, то результат вызова метода предполагается зависящим от типа аккаунта (демо/не демо); для различных типов аккаунта кэшируются разные данные. При снятии отметки возвращаемый результат от типа аккаунта не зависит.
Depends on region (RU, KZ, UZ)	Если флаг отмечен, то результат вызова метода зависит от страны, из которой выполнен вызов (например, RU, KZ, UZ); для различных стран кэшируются разные данные. При снятии отметки возвращаемый результат от страны не зависит.



Атрибуты **Depends on demo/non-demo client** и **Depends on region (RU, KZ, UZ)** актуальны для глобального кэширования.

Кэширование результатов можно задать для:

- [произвольного метода](#);
- [области видимости](#);
- [проху-метода](#);
- [декларативного](#) списочного метода;
- списочного метода, [реализуемого вручную](#).

Инвалидация кэша

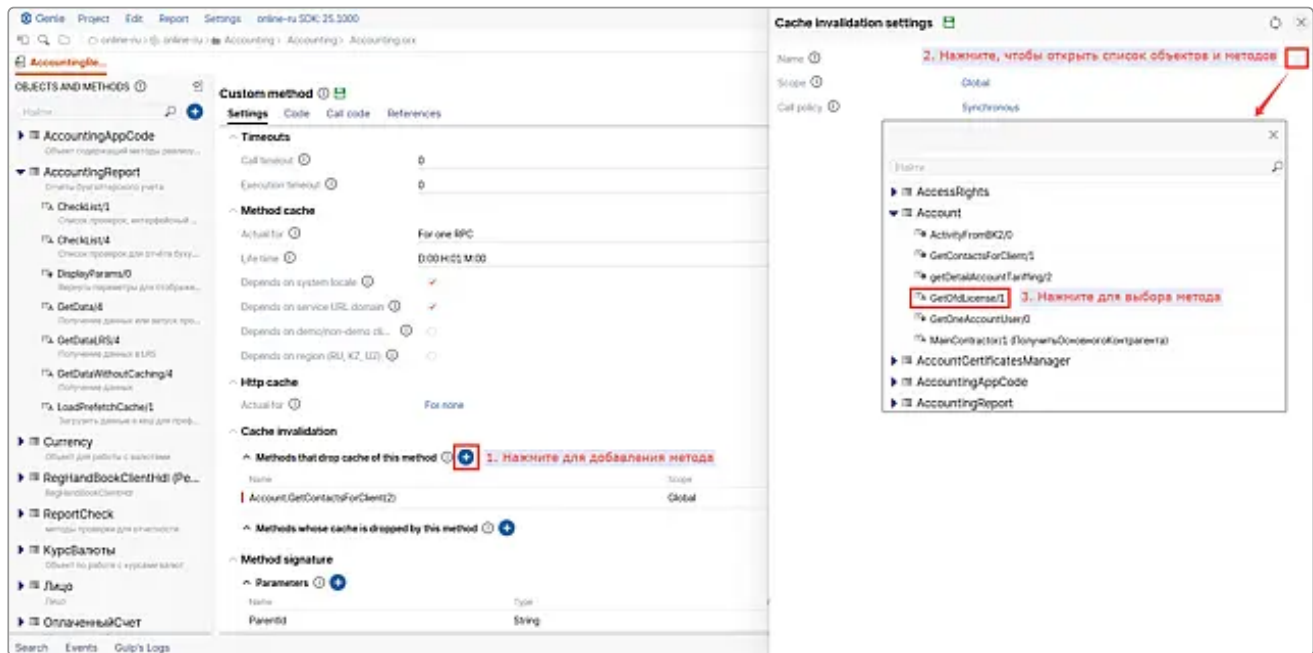
Результат, возвращаемый методом БЛ, кэшируется на срок, заданный в настройках такого метода БЛ в атрибуте **Life time**.

Если закэшированный результат устаревает раньше предполагаемого срока, то он должен быть принудительно сброшен с помощью следующих механизмов:

- [Автоинвалидация кэша;](#)
- [Принудительная очистка кэша;](#)
- [Обновление кэша при вызове метода;](#)
- [Прямое обращение в кэш.](#)

Автоинвалидация кэша

Механизм, позволяющий сбрасывать результаты исполнения одного метода(-ов) при вызове другого метода(-ов).

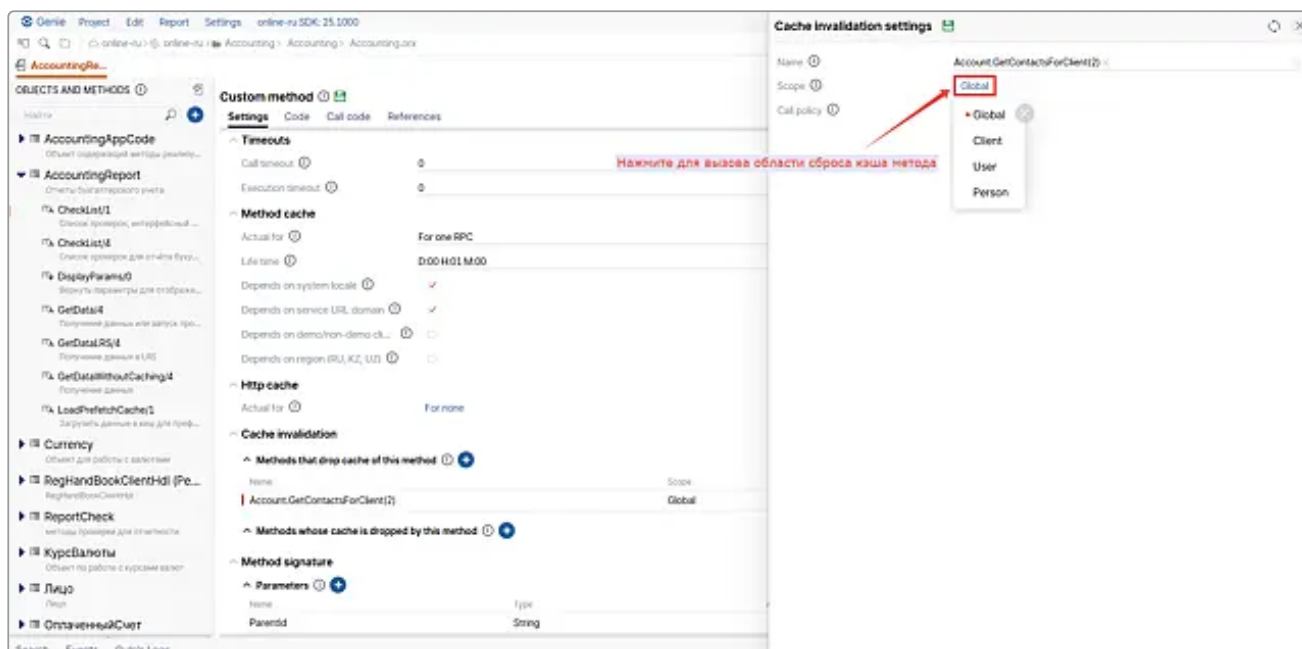


В настройках автоинвалидации кэша **Cache Invalidation** можно задать область сброса кэша метода (**Scope**) и политику вызова метода инвалидации (**Call policy**).

Область сброса кэша метода

Указание области сброса кэша метода (Scope) позволяет выполнять инвалидацию кэша:

- **Global** - глобально, по всем результатам исполнения метода;
- **Client** - по текущему клиенту, под которым выполняется метод;
- **User** - по текущему пользователю;
- **Person** - по текущей персоне.



Например, если вызванный метод выполняется под пользователем 1234, то будут удалены только те результаты исполнения инвалидируемого метода, которые относятся к пользователю 1234 (были получены при вызове метода из-под аккаунта этого пользователя). Аналогично по клиенту и персоне.

Если область сброса кэша метода указана **Global**, то при вызове метода, сбрасывающего кэш, кэш сбрасываемого метода очищается полностью.

Если область сброса кэша метода указана **Client** или **User**, но при вызове метода, сбрасывающего кэш, по каким-либо причинам не задан идентификатор сессии, то очистка кэша для сбрасываемого метода выполнена не будет.

Если область сброса кэша указана **Person**, но при вызове метода, сбрасывающего кэш, не задан HTTP-заголовок X-SPID (идентификатор персоны, должен быть типа UUID, может задаваться как заголовок сессии), то очистка кэша для сбрасываемого метода выполнена не будет.

Если при области сброса кэша метода **Client**, **User** или **Person** соответствующий атрибут сессии (идентификатор сессии или заголовок X-SPID) корректно задан, то будет выполнена очистка кэша сбрасываемого метода по соответствующей области.

Области сброса кэша метода **User** и **Client** вложены одна в другую: **User** — более узкая, **Client** — более широкая, одному клиенту может соответствовать несколько пользователей.

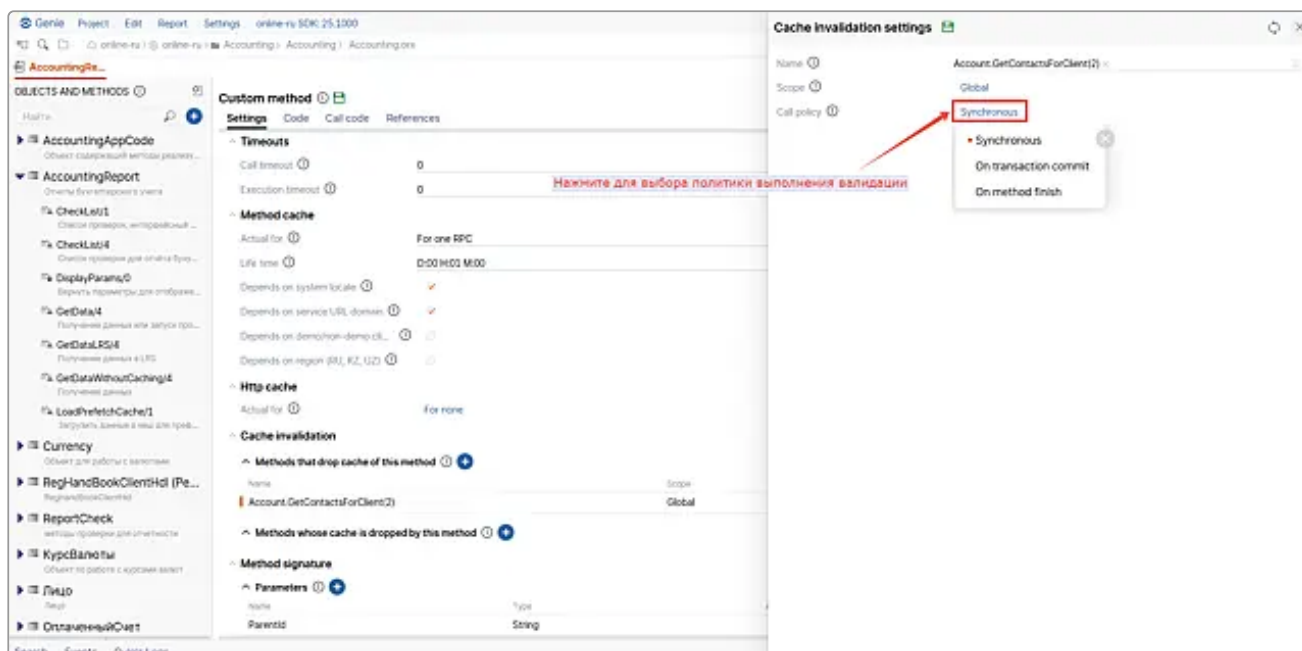
Область сброса кэша **Person** не включена в иерархию "Пользователь" → "Клиент", разным персонам могут соответствовать одинаковые клиенты/пользователи и наоборот.

При этом область сброса **Global** охватывает все остальные области сброса. Области сброса кэша соответствуют одноименным областям актуальности кэшируемых данных.

Политика вызова метода инвалидации

Политика выполнения инвалидации позволяет задать момент времени инвалидации кэша:

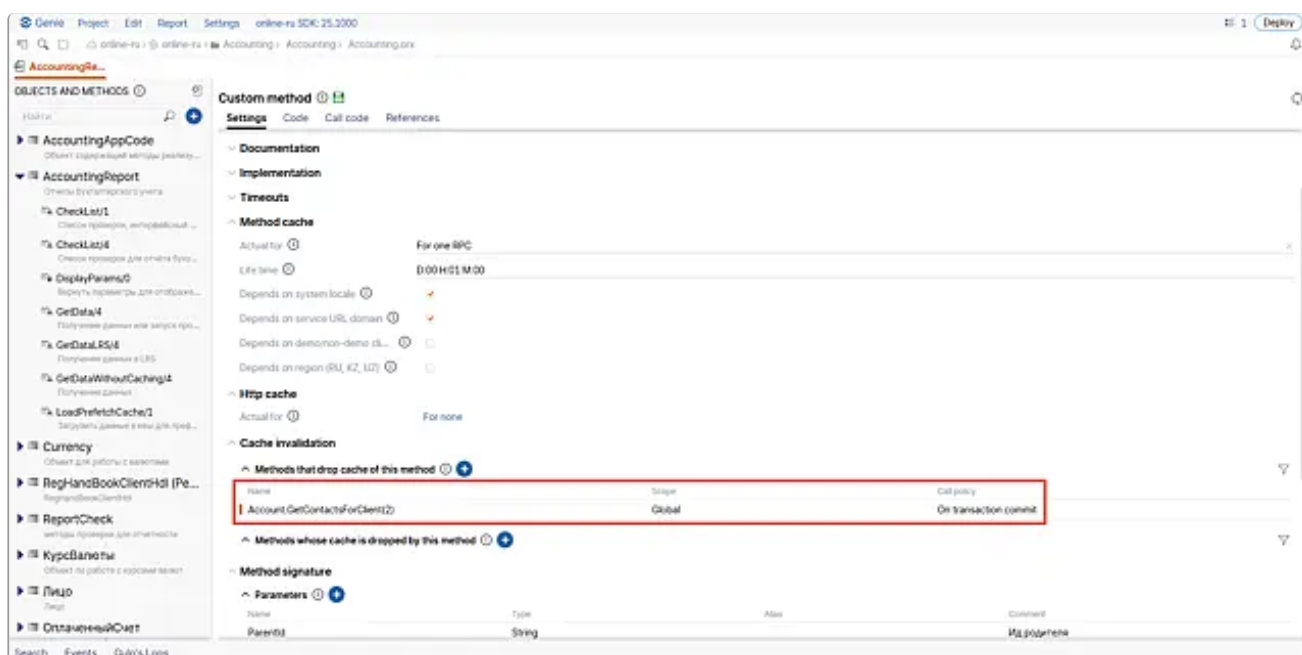
- **Synchronous**: синхронно, непосредственно после завершения работы исполняемого метода;
- **On transaction commit**: по коммиту текущей транзакции. Если нет активной транзакции, инвалидация производится синхронно. Если происходит откат транзакции, инвалидация кэша не выполняется.
- **On method finish**: после завершения текущего RPC-вызова.



Использование автоинвалидации по коммиту транзакции и по завершению текущего RPC-вызова позволяет уменьшить количество вызовов методов очистки кэша. Это особенно актуально для случаев, когда в процессе одного RPC-вызова производится многократный вызов другого метода, в котором выполняется очистка кэша. Однако немедленная очистка не требуется.

В ситуациях, когда очистку кэша сбрасываемых методов следует выполнить после успешного завершения транзакции при работе с БД, политику вызова при автоинвалидации следует задать равной **On transaction commit** — по коммиту текущей транзакции.

В этом случае многократные вызовы методов инвалидации кэша, выполненные во время транзакции, будут объединены в один, который будет выполнен по коммиту транзакции. Соответственно, при откате транзакции очистки кэша не будет.



Пример 1. Если при выполнении внешнего RPC вызова "МойМодуль.ОбновитьКонфигурацию" производится многократный локальный вызов методов, для которых настроена автоинвалидация кэша — например,

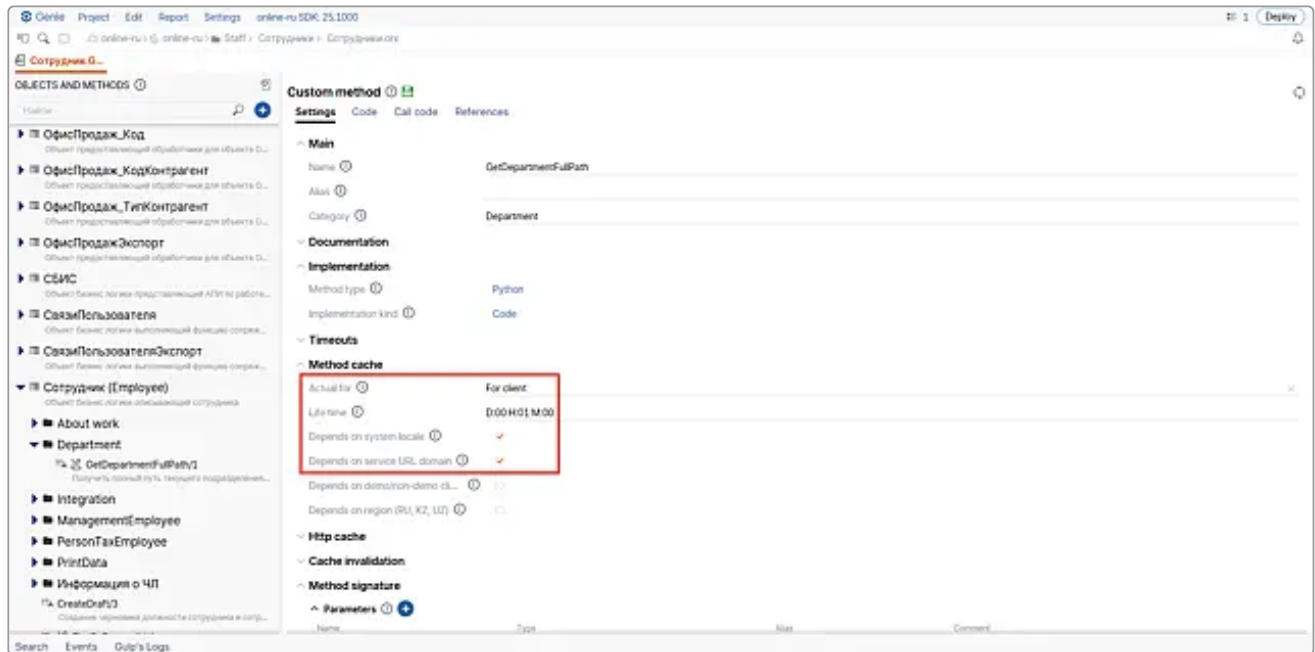
- 1 МойПараметр.Установить("название1", "значение1")
- 2 ...
- 3 МойПараметр.Установить("названиеN", "значениеN")

где метод "МойПараметр.Установить" сбрасывает кэш метода "МойПараметр.Прочитать", то во многих случаях предпочтительно задать политику вызова автоинвалидации равной **On method finish** — после завершения текущего RPC-вызова.

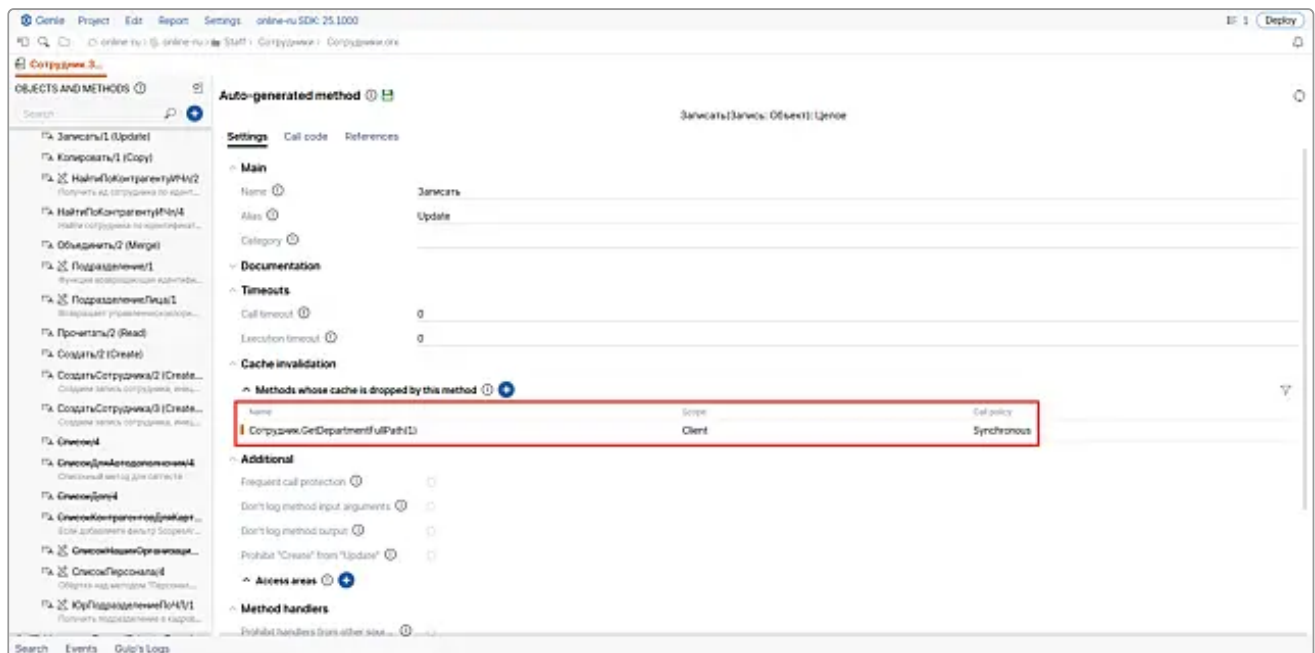
При этом вместо многократных повторяющихся вызовов методов инвалидации кэша во время выполнения метода "МойМодуль.ОбновитьКонфигурацию" все эти вызовы будут объединены в один, который будет выполнен

после завершения RPC вызова, т.е. по окончании исполнения метода "МойМодуль.ОбновитьКонфигурацию".

Пример 2. В сервисном модуле Сотрудники для объекта **Сотрудники** описан метод GetDepartmentFullPath, который возвращает полный путь текущего подразделения сотрудников. Поскольку значение текущего подразделения меняется достаточно редко, для этого метода настроено кэширование:



Однако данные по подразделению лица могут измениться, поэтому для отображения актуальных данных в методе Записать настроено обновление кэша:



Сбрасываем кэш метода для текущей организации, к которой относится сотрудник, при этом другие организации не затрагиваем: результаты кэширования данного метода для других клиентов не сбрасываются, поэтому для них заново вызывать метод не потребуется, за счет этого экономится время и вычислительные ресурсы.

Принудительная очистка кэша

Механизм, позволяющий производить немедленный сброс результатов исполнения метода(-ов).

Чтобы сбросить кэш метода/методов, необходимо создать запрос на очистку кэша [InvalidationRequest](#) (Python/C++), указать критерии сброса кэша с помощью методов класса [InvalidationRequest](#) и отправить запрос системе кэширования с помощью метода [ClearCacheByInvalidationRequest](#) (Python) или [ClearByInvalidationRequest](#) (C++).

Python:

```
1 import bl_cache
2 ir = bl_cache.InvalidationRequest()
3 ir.AddMethod('Example.Method1', 0)
```

```

4  ir.AddMethodByClient('Example.Method2', 5, 12345)
5  bl_cache.ClearCacheByInvalidationRequest(ir) # отправка запроса

```

C++:

```

1  #include <sbis-bl-cache/cache_cleaner.hpp>
2  #include <sbis-cache-datamodels/invalidation_request.hpp>
3  blc::InvalidationRequest ir;
4  ir.AddMethodToInvalidate( L"Example.Method1", 0 );
5  ir.AddMethodToInvalidateByClient( L"Example.Method2", 5, 12345 );
6  blc::Cleaner::ClearByInvalidationRequest( ir );

```

Также для очистки кэша вы можете использовать методы БЛ, которые принадлежат объекту БЛ **КэшированиеМетодов**.

Методы очистки кэша при успешной очистке возвращают целочисленное значение 0. В случае, если при очистке кэша произошла ошибка, например, вызов был прерван по таймауту — возвращается ненулевое значение.

КэшированиеМетодов.ОчиститьКэш (MethodCache.ClearAllResults)

Входные параметры	Описание
- ИмяМетода: Строка - ЧислоПараметров: Целое	Очищает кэш одного метода

КэшированиеМетодов.ОчиститьКэшКлиента (MethodCache.ClearClientResults)

Входные параметры	Описание
- ИдентификаторКлиента:Целое - ИмяМетода: Строка - ЧислоПараметров: Целое	Очищает кэш одного метода по указанному клиенту.

КэшированиеМетодов.ОчиститьКэшМетодов (MethodCache.ClearAllResultsMultiple)

Входные параметры	Описание
СписокМетодов: Набор записей. Формат записи: - ИмяМетода: Строка - ЧислоПараметров: Целое	Очищает кэш нескольких методов, перечисленных в RecordSet.

КэшированиеМетодов.ОчиститьКэшПользователейКлиента (MethodCache.ClearClientUsersResults)

Входные параметры	Описание
- ИдентификаторКлиента: Целое - ИдентификаторыПользователей: Целочисленный массив - ИмяМетода: Строка - ЧислоПараметров: Целое	Очищает кэш одного метода для нескольких или всех пользователей клиента.

КэшированиеМетодов.ОчиститьКэшПользователя (MethodCache.ClearUserResults)

Входные параметры	Описание
- ИдентификаторПользователя: Целое - ИмяМетода: Строка - ЧислоПараметров: Целое	Очищает кэш одного метода по указанному пользователю.

КэшированиеМетодов.ОчиститьКэшПоНаборуПараметров (MethodCache.ClearResultsPartialRecord)

Входные параметры	Описание
- ИмяМетода: Строка - ЧислоПараметров: Целое - Параметры: Запись. Аргументы в Record задаются в порядке их следования в сигнатуре метода. При этом перечислять можно один, два или три первых параметра.	Удаляет результаты исполнения метода, соответствующие значениям нескольких первых его аргументов. Рассмотрим принцип работы метода. Есть метод <code>My.Method(a1, a2, a3, a4)</code>, где все аргументы типа <code>String</code>. Вызовем его несколько раз с разными параметрами: <pre>1 My.Method("s1_1", "s2_1", "s3_1", "s4") 2 My.Method("s1_1", "s2_1", "s3_2", "s4") 3 My.Method("s1_1", "s2_2", "s3_1", "s4") 4 My.Method("s1_1", "s2_2", "s3_2", "s4") 5 My.Method("s1_2", "s2_1", "s3_1", "s4") 6 My.Method("s1_2", "s2_1", "s3_2", "s4") 7 My.Method("s1_2", "s2_2", "s3_1", "s4") 8 My.Method("s1_2", "s2_2", "s3_2", "s4")</pre> При этом в кэш запишутся 8 результатов выполнения метода с соответствующими наборами аргументов. Очистка кэша по набору параметров удаляет результаты выполнения метода со значениями аргументов, соответствующими указанным в наборе. Например: вызов <pre>1 o = BLObject("КэшированиеМетодов") 2 o.Invoke("ОчиститьКэшПоНаборуПараметров", "My.Method", 4, Record ({ "a1" : "s1_1" }))</pre> удалит из кэша результаты (1), (2), (3), (4). А следующий код вызовет очистку кэша <pre>1 r = Record() 2 r.AddString("a1", "s1_2") 3 r.AddString("a2", "s2_2") 4 o = BLObject("КэшированиеМетодов") 5 o.Invoke("ОчиститьКэшПоНаборуПараметров", "My.Method", 4, r)</pre> для результатов (7), (8). Удаление производится только по одному, двум или трем первым аргументам метода.

[deprecated] КэшированиеМетодов.ОчиститьКэшПоХэшу (MethodCache.ClearHashResults)

Входные параметры	Описание
- Хэш:Строка — передается строковое представление хэша всей совокупности аргументов метода. - ИмяМетода: Строка - ЧислоПараметров: Целое	Удаляет один конкретный результат исполнения метода по хэшу его аргументов.

Любой из перечисленных методов можно вызвать напрямую из C++. Пример:

```
blc::Cleaner::ClearAllResults( L"Object.Method", num_args );
```

Методы ОчиститьКэшПоХэшамПараметров и ОчиститьКэшПоНаборуПараметров работают одинаково, они отличаются только представлением входных данных — для длительного хранения большого числа наборов аргументов с целью последующей очистки, возможно, имеет смысл хранить хэши, а не Record'ы.



Вместо методов ClearAllPersonResults и ClearAllUserResults можно использовать ClearUserResults и ClearPersonResults для очистки кэша по конкретному методу.

Вместо ClearResultsPartial следует использовать blc::Cleaner::ClearResultsPartial либо blc::Cleaner::ClearResultsPartialArgs (только для C++).



Данные методы вычисления хэшей доступны только из C++. Удалить из кэша один результат по конкретному полному набору аргументов метода из Python сейчас невозможно.

Обновление кэша при вызове метода

Механизм, позволяющий обновить некоторый конкретный закэшированный результат исполнения метода, запретив чтение из кэша на время данного конкретного вызова метода. Такой запрет налагается посредством добавления к HTTP запросу заголовка X-Method-Cache-Control: no-cache.

- Пример для C++

```
1 blcore::EndPoint ep( L"My Service" );
2 blcore::Object obj( L"MyObject", ep );
3
4 // Отключаем чтение из кэша
5 ep.SetHeader( L"X-Method-Cache-Control", L"no-cache" );
6
7 // Метод вызовется с отключенным чтением из кэша, но будет выполнена
  запись в кэш
8 Type result = obj.Invoke< Type >( L"MyMethod" );
9
10 // После вызова метода важно не забыть вновь включить чтение из кэша
11 ep.SetHeader( L"X-Method-Cache-Control", L"" );
12
13 // Теперь при вызове метода чтение из кэша будет выполнено
14 Type result = obj.Invoke< Type >( L"MyMethod" );
```

- Пример для Python

```
1 ep = sbis.EndPoint( "My Service" )
2 obj = sbis.BLObject( "MyObject", ep )
3
4 # Отключаем чтение из кэша
5 ep.SetHeader( "X-Method-Cache-Control", "no-cache" )
```

```
6
7 # Метод вызовется с отключенным чтением из кэша, но будет выполнена
  запись в кэш
8 result = obj.Invoke( "MyMethod" )
9
10 # После вызова метода важно не забыть вновь включить чтение из кэша
11 ep.SetHeader( "X-Method-Cache-Control", "" )
12
13 # Теперь при вызове метода чтение из кэша будет выполнено
14 result = obj.Invoke( "MyMethod" )
```

Прямое обращение в кэш

Обычно в кэш обращается вызываемый сервис. Представим, что сервис А вызывает сервис В, на котором метод кэшируемый. Перед тем как дать управление методу вызвать его, сервис В обращается в кэш и [проверяет в нем наличие ответа на данный метод](#).

В платформе реализована оптимизация этого процесса, т.е. если А и В - платформенные сервисы, то возможна упрощенная механика - прямое обращение в кэш.

В этом случае сервис А перед первым вызовом метода на сервисе В запрашивает с него метаданные о вызываемом методе. В этих метаданных может быть указано, что метод кэшируемый. Тогда если хранилища кэшированных данных сервисов А и В совпадают, то сервис А оказывается способен сам выполнить обращение в кэш сервиса В.

Данная оптимизация возможна, если сервис А знает, куда будет передан его вызов. Если сервис В является мультиверсионным или мультиарендным, то заранее нельзя знать, куда мы передадим вызов. Соответственно, в этом случае прямые обращения в кэш не выполняются.

Тем не менее, иногда эта оптимизация желательна, т.к. если отправить весь трафик на сервис В, он не выдержит поток трафика, который требуется раздать. Для решения данной проблемы существуют параметры, которые позволяют включить механику прямого обращения в кэш даже для мультиверсионных сервисов:

- **Ядро.Кэширование методов.AllowedMultiversionTargets** - список названий мультиверсионных сервисов, при обращении к которым прямое чтение из кэша со стороны клиента разрешено.
- **Ядро.Кэширование методов.MultiversionDirectRead** - включает отдачу информации о кэшировании для мультиверсионного сервиса для работы прямого обращения в кэш со стороны клиента. Прямое обращение в кэш будет работать со стороны тех клиентов, для которых данный мультиверсионный сервис входит в число разрешенных.

Примеры:

- на сервисе online (вызываемый, multiversion) указывается Ядро.Кэширование методов.MultiversionDirectRead = Да
- на сервисе представления (вызывающий) указывается Ядро.Кэширование методов.AllowedMultiversionTargets = ["online"]

При такой конфигурации будет работать прямое обращение в кэш при вызовах с [сервиса представления](#) на сервис online.

Специализированные сервисы кэширования

Ресурсы сервиса кэширования ограничены. В случае, если прикладной сервис (или группа сервисов) активно использует кэширование, сервис кэширования может не справиться с нагрузкой. От этого страдают как сервис-источник нагрузки, так и остальные сервисы. В этом случае можно поднять отдельный сервис кэширования для сервиса (группы сервисов).

Для добавления и настройки нового сервиса кэширования свяжитесь с сотрудниками участка [Сервис кэширования](#).

При сбросе кэша какого-либо метода, он сбрасывается только в рамках одного сервиса кэширования. Например, есть два сервиса - А и Б, и они используют один сервис кэширования, тогда сервис Б может сбрасывать кэш сервиса А. Сброс выполняется независимо от принадлежности метода сервису.

При применении специализированных сервисов кэширования это не работает. Если сервис А использует один сервис кэширования, а сервис Б - другой, то сервис Б не может сбрасывать кэш сервиса А.

При использовании специализированных сервисов теряется сквозная инвалидация. Чтобы ее использовать, необходимо применить методы [ClearAllCacheStoragesByInvalidationRequest](#) (Python) или [ClearAllStoragesByInvalidationRequest](#) (C++), которые посылают запрос инвалидации на все известные сервисы кэширования.

Админка сервиса кэширования

[Админка сервиса кэширования](#) позволяет визуализировать обобщенные характеристики данных, хранящихся в сервисе кэширования, наблюдать за текущим состоянием сервиса кэширования и историей его изменения, а также сбросить кэш метода.

Кэш

36.2000-12 от 26.03.2026

Сессия Роман Вадимович

Общий кэш

Поиск параметров кэша

Название	Ссылки, кб			Название кэша	% от общего
	Общий	В памяти	На диске		
Кэш feed	0.0	0.0	0	1	0.00
AFBCMonitoring.GetAll/0	13.2	13.2	0	770	83.31
AUSNCalc.GetType/0	1.8	1.8	0	107	0.37
AUSNOperation.GetType/0	0	0	0	0	0.00
AbonentBox.ListIgnoredInspection/0	1369.2	1369.2	0	49775	90.38
AbonentProps.AddOrGet/1	27.6	27.6	0	1326	63.91
AccessArea.CameraOrRead/0	20.3	20.3	0	39	24.47
AccessRights.CountUsersAccess/2	13.3	13.3	0	28	61.77
AccessRights.UserCountryBySpecialStatus/1	393.3	393.3	0	669	36.19
AccessRights.UsersAccessCache/3	2115.1	2115.1	0	100067	97.93
Accommodation.GetWorkInterval/1	3822.5	2933.8	888.7	1323	78.27
Account.Countries/1	0.1	0.1	0	3	0.26
Account.CounterCached/1	5.7	5.7	0	267	78.28
Account.FindByProps/3	876.1	876.1	0	36734	47.46
Account.FindByPropsFromSp/3	19077.8	19077.8	0	610376	69.02
Account.GetName/1	3360.7	3360.7	0	32898	71.00
Account.GetNameByClientOr/1	8.6	8.6	0	77	71.45
Account.GetOneAccountUser/0	166.3	166.3	0	81	8.66
Account.GetPurchaseServices/2	18981.1	18981.1	0	33344	56.29
Account.OperatorActivity/1	28.9	28.9	0	74	25.94
Account.Parameters/0	9240.3	9240.3	0	20185	96.95
Account.Read/1	2612.1	2612.1	0	5699	95.48
Account.UnassignedCounter/1	0.0	0.0	0	1	0.74
Account.ApplicationConfigurations.ComponentsState/3	5579.2	5579.2	0	2363	98.42
Account.ApplicationConfigurations.List/2	11175.1	11175.1	0	1507	28.86

[Подробнее о работе админки](#)